

Python–n8n Integration and Final Workflow

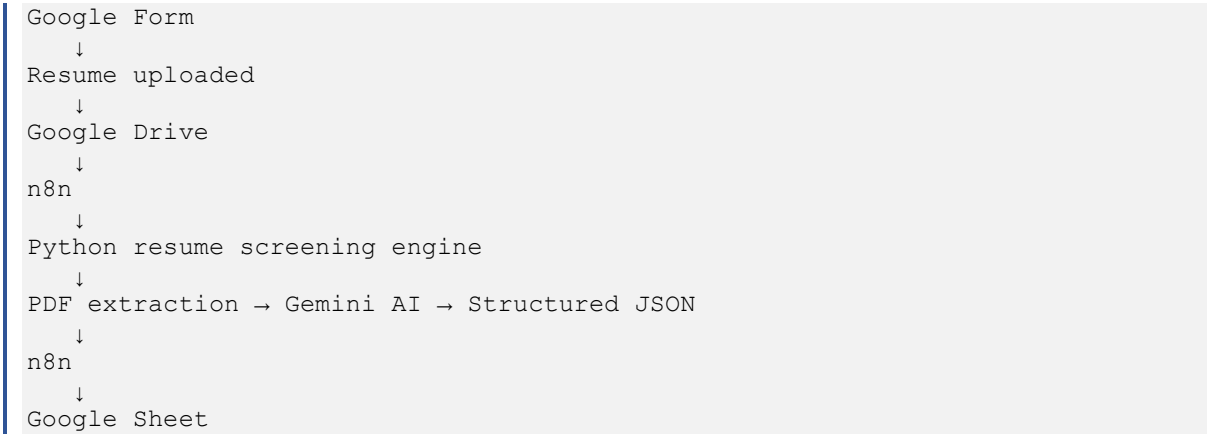
1. Overview

This document explains how the parts of the AI Resume Screening System are connected: n8n, the Python Flask API, the screening engine, Gemini AI, result storage and the Streamlit dashboard. It also records how the design changed from the original plan to the final implementation.

Related documents: **Python_Implementation.docx** and **Ubuntu_WSL_Environment.docx**.

2. Original Planned Architecture

The original design used n8n to pass a resume from Google Drive to a Python runner, and store the returned result in Google Sheets:



In this design, Python communicated with n8n through `n8n_runner.py`, which accepted JSON input and returned a JSON response.

3. What Changed

Area	Original design	Final implementation
n8n–Python communication	JSON input/output through <code>n8n_runner.py</code>	HTTP POST request to the Flask API endpoint <code>/screen</code>
Resume transfer	Resume read from a file path	PDF sent as binary data in the request and saved temporarily by Python
Result storage and display	Google Sheets (planned)	<code>data/candidates.json</code> , displayed in a Streamlit HR dashboard

Area	Original design	Final implementation
Where n8n runs	Not specified	Docker container, reaching Python through the WSL IP address
Python component	Command-style engine	Flask API wrapping the original engine

4. Final Architecture

```

n8n (running in Docker)
  ↓ HTTP POST: resume (PDF) + job_description
Flask API on Ubuntu/WSL (http://172.19.95.137:5000/screen)
  ↓
Temporary resume file
  ↓
Screening engine: PDF extraction (pdfplumber)
  ↓
Gemini AI analysis
  ↓
Validated structured JSON result
  ↓
n8n extracts result fields
  ↓
data/candidates.json
  ↓
Streamlit HR dashboard (http://localhost:8501)

```

The earlier stages of the flow (Google Form, Google Drive) belong to the original design. The later documentation focuses on the part from n8n onwards.

5. Role of Each Component

Component	Role
n8n	Automates the workflow. Sends the resume and job description to Python and processes the returned result fields.
Docker	Runs the n8n instance.
Flask API (<code>api.py</code>)	Receives requests from n8n, saves the resume temporarily and runs the screening engine. Endpoints: <code>/screen</code> , <code>/result</code> , <code>/health</code> .
Screening engine	Extracts resume text, sends it with the job description to Gemini, and validates the returned JSON.
Gemini AI	Evaluates the resume against the job description.
<code>data/candidates.json</code>	Stores the screening results.
Streamlit (<code>app.py</code>)	Displays the stored results to HR.
Ubuntu/WSL	Hosts the Python backend and the dashboard.

6. Request and Response Details

6.1 Request from n8n

n8n sends an HTTP POST request to `/screen` with two inputs:

```
resume          → PDF file (binary)
job_description → Job requirements (text)
```

In the n8n HTTP Request node, the PDF is sent as binary data using Input Data Field Name: `data`. Python reads it with `request.files.get("resume")`.

6.2 Response from Python

Python returns a JSON response containing:

- `candidate_name`
- `match_score`
- `key_strengths`
- `missing_skills`
- `final_verdict`
- `explanation`

If something goes wrong (for example, no resume is received), Python returns the standard error structure with `"success": false` and an error message.

6.3 Result handling

n8n extracts the individual result fields so they can be stored and displayed easily. The Python backend stores each screening result in `data/candidates.json`, which the Streamlit dashboard reads to show candidate results to HR.

7. Docker-to-WSL Connection

n8n runs inside Docker while the Flask server runs in Ubuntu/WSL. From inside the container, `localhost:5000` does not refer to the Flask server, so n8n uses the WSL network IP address instead:

```
http://172.19.95.137:5000/screen
```

This allows two applications running in different environments to communicate.

8. Integration Problems and Solutions

Problem	Solution	Stage
n8n sent only the filename or metadata, not the PDF	Configured the HTTP Request node to send binary data with Input Data Field Name: <code>data</code> .	Later
<code>localhost:5000</code> from n8n did not reach the Flask server	Used the WSL IP address with port 5000.	Later
Result needed to be structured before dashboard use	n8n extracts the individual fields from the JSON result.	Later

Problem	Solution	Stage
Free-form AI output was hard for n8n to process	Gemini prompt changed to return a fixed JSON structure.	Original
Incorrect file and data paths	Corrected to the actual project directory paths.	Later

9. My Contribution vs Later Team Integration

My original contribution	Later modifications by teammate
Fixed JSON output format usable by n8n	Flask API and HTTP-based n8n communication
n8n_runner.py interface with validation and standard errors	Binary PDF transfer configuration in n8n
Screening engine (PDF extraction, Gemini, validation)	Docker-to-WSL connection
Original planned architecture	Result storage in JSON and Streamlit dashboard

10. Current Limitations

- The connection relies on a fixed WSL IP address.
- The system runs on a personal computer and must be started manually.
- Results are stored in a JSON file rather than a database.
- The API has no authentication.

11. Future Scope

- Deploy Flask and n8n on a Linux server or cloud VM using Docker containers
- Replace the JSON file with a database
- Add authentication, validation and logging to the API
- Add HTTPS and monitoring
- Support batch processing and OCR for scanned resumes

12. Final Outcome

The project evolved from a Python runner designed for n8n into a working prototype in which n8n, running in Docker, sends resumes to a Flask API on Ubuntu/WSL. The screening engine analyses each resume with Gemini AI, and the structured results are stored and displayed to HR through a Streamlit dashboard. The system is still a prototype: it runs on a development machine and has considerable scope for future improvement.